Server ja Node.js

11IT · Veebirakendused · seitsmes tund · konspekt

> **Selle tunni suur mõte:** server on lihtsalt programm, mis ootab küsimusi ja vastab neile. Täna kirjutad sellise ise. See tuleb umbes kümme rida pikk.

Mis juhtub, kui avad veebilehe?

Kirjutad brauserisse `tahvel.edu.ee` ja vajutad Enterit. Sekundi pärast on leht ees. Selle sekundi jooksul juhtus kaks asja.

Kõigepealt saatis sinu brauser kuhugi kaugele arvutisse küsimuse: „palun anna mulle see leht“. Seda küsimust nimetatakse **päringuks**. Seal kaugel arvutis töötab programm, mis päringu kätte sai, otsis õige asja üles ja saatis selle tagasi. Seda, mis tagasi tuleb, nimetatakse **vastuseks**.

Programmi, mis päringutele vastab, nimetatakse **serveriks**. Seda, kes küsib (meil brauser), nimetatakse **kliendiks**.

Restoranis	Veebis
Klient istub laua taga ja tellib	**Klient** ehk brauser
Tellimus: „üks hernesupp, palun“	**Päring**
Köök, kus tellimus täidetakse	**Server**
Supitaldrik, mis lauale tuuakse	**Vastus**

Pane tähele, et köök ei tee midagi enne, kui keegi tellib. Samamoodi server. Ta käivitub, ütleb „olen valmis“ ja jääb ootama. Võib-olla tuleb esimene päring kohe, võib-olla tunni pärast. Iga päringu peale teeb ta oma töö ja ootab jälle edasi.

Üks server on sul tegelikult juba olnud. Kui kirjutasid Reacti projektis `npm run dev`, ei tulnud terminali uut `PS …>` rida. See oli sellepärast, et Vite on server ja ta jäi ootama. `Ctrl+C` paneb serveri kinni, nagu köögi uks lukku.

Milleks meil server?

Söökla rakenduses olid arvustused failis `andmed.ts` ja uued arvustused ainult brauseri mälus. Vajutasid F5 ja kõik uued arvustused olid kadunud. Pinginaaber ei näinud neid kunagi, sest need olid ainult sinu arvutis.

Server lahendab selle. Kui arvustused on serveris, küsivad kõik brauserid neid samast kohast. Sinu lisatud arvustust näeb ka pinginaaber. Täna teeme esimese sammu: server, mis annab arvustused välja.

Node.js: JavaScript ilma brauserita

Seni käivitas sinu JavaScripti alati brauser. Serveris pole aga brauserit, pole akent ega nuppe. Seal käivitab JavaScripti programm nimega **Node.js**. Keel on täpselt sama: muutujad, funktsioonid ja massiivid töötavad nagu ennegi.

Kaks erinevust tuleb meelde jätta. Esiteks `console.log` ei kirjuta enam F12 konsooli, vaid **terminali**. Teiseks pole Node’is lehte, nii et `document` ja `querySelector` ei tööta. Neid ei ole serveris vaja.

Node'i programm käivitatakse terminalis nii:

```
node index.js
```

JSON: andmed tekstina

Server võib saata vastuseks HTML-lehe, aga meie server saadab andmeid. Andmed liiguvad serverist brauserisse kujul, mida nimetatakse **JSON**-iks. See näeb välja peaaegu nagu JavaScripti massiiv ja objektid:

```
[
  { "id": 1, "toit": "Kanakaste riisiga", "hinne": 5 },
  { "id": 2, "toit": "Hernesupp", "hinne": 3 }
]
```

Nurksulud on massiiv, loogelised sulud on üks objekt. Ainus erinevus JavaScriptist on see, et JSON-is on ka väljade nimed jutumärkides. Seda ei pea ise kirjutama, Express teeb selle meie eest.

Pakett ja npm

Serveri saaks kirjutada ka puhta Node'iga, aga see on pikk ja tülikas. Seepärast kasutame **Expressi**. Express on **pakett** ehk kellegi teise kirjutatud kood, mille saad oma projekti ühe käsuga:

```
npm install express
```

`npm` on programm, mis toob pakette internetist. Pärast seda käsku on kaustas kaks uut asja:

- `node_modules` on kaust, kus on Express ja kõik, mida Express ise vajab. Seal on palju kaustu. Ära ehmu ja ära sinna midagi muuda;
- `package.json`-isse tekib rida `"express"`. See on nimekiri: „sellel projektil on vaja Expressi".

Tänu sellele nimekirjale ei pea `node_modules`-it kellelegi saatma. Kes projekti saab, kirjutab `npm install` ja npm toob kõik ise.

Aadress

Meie server töötab sinu enda arvutis. Tema aadress on selline:

```
http://localhost:3000/api/arvustused
```

Osa	Mida tähendab
`localhost`	See arvuti, mille taga sa istud. Server ei ole internetis, vaid sinu juures.
`3000`	**Port** ehk ukse number. Ühes arvutis võib töötada mitu serverit ja igaüks on oma ukse taga. Reacti Vite on uksel 5173, meie server on uksel 3000.
`/api/arvustused`	Mida küsitakse. Ühel serveril võib olla palju aadresse, igaühe taga on midagi muud. `/api` alguses on kokkulepe: siit tulevad andmed.

Esimene server

Terve server mahub kümnesse ritta. Loe see läbi ja siis rida-realt seletus:

`server/index.js`

```
const express = require('express');

const app = express();

app.get('/', (req, res) => {
  res.send('Tere! Söökla server töötab.');
});

app.listen(3000, () => {
  console.log('Server töötab: http://localhost:3000');
});
```

Kood	Mida see teeb
`const express = require('express');`	Toob Expressi paketi sisse. `require` tähendab „võta see pakett kasutusse“.
`const app = express();`	Teeb uue serveri ja paneb selle muutujasse `app`.
`app.get('/', (req, res) => {`	Reegel: kui keegi küsib aadressi `/`, käivita see funktsioon. `get` tähendab, et klient tahab midagi *saada*.
`(req, res)`	Funktsioon saab kaks asja. `req` on päring ehk see, mida küsiti. `res` on vastus ehk sellega saadad midagi tagasi.
`res.send('Tere! ...');`	Saadab vastuseks teksti.
`app.listen(3000, () => {`	Paneb serveri uksel 3000 käima ja ootama.
`console.log('Server töötab...');`	Kirjutab terminali, et server on valmis.

Reeglit „see aadress käivitab selle funktsiooni“ nimetatakse **marsruudiks**. Iga uue aadressi jaoks teed uue marsruudi, kõik täpselt sama kujuga.

Andmed JSON-ina

Teksti saatmine on tore, aga meie tahame saata arvustusi. Paneme serverisse massiivi ja teeme sellele marsruudi:

```
const arvustused = [
  { id: 1, toit: 'Kanakaste riisiga', hinne: 5, kommentaar: 'Parim toit sel nädalal!' },
  { id: 2, toit: 'Hernesupp', hinne: 3, kommentaar: 'Natuke liiga paks.' },
  { id: 3, toit: 'Makaronid hakklihaga', hinne: 4, kommentaar: 'Klassika, alati hea.' },
];

app.get('/api/arvustused', (req, res) => {
  res.json(arvustused);
});
```

Ainus uus asi on `res.json`. See teeb massiivist JSON-i ja saadab selle. Ava brauseris `http://localhost:3000/api/arvustused` ja näed oma arvustusi JSON-ina.

Kaks käsku, mida vajad kogu aeg

Käsk	Mida teeb
`node --watch index.js`	Paneb serveri käima. `--watch` tähendab: kui salvestad faili, käivitab server end ise uuesti. Ilma selleta töötaks edasi vana kood.
`Ctrl+C`	Paneb serveri kinni. Terminal on jälle vaba.

Kui midagi ei tööta

Serveri vead ei tule F12 konsooli, vaid **terminali**, kus server töötab. Vaata alati kõigepealt sinna.

Mida näed	Mida see tähendab	Mida teha
`Cannot find module 'express'`	Selles kaustas pole Expressi.	Kontrolli `pwd`-ga, et oled kaustas `server`. Siis `npm install express`.
`EADDRINUSE` … `3000`	Uks 3000 on juba kinni. Sinu server töötab juba teises terminalis.	Leia see terminal ja vajuta `Ctrl+C`.
`require is not defined`	Sinu `package.json`-is on rida `"type": "module"`.	Kustuta see rida `package.json`-ist.
`Cannot GET /api/…` brauseris	Sellist marsruuti pole.	Võrdle aadressi ja `app.get` rida täht-tähelt.
Brauser ütleb, et lehte ei saa avada	Server ei tööta.	Vaata terminali. Kas server käib või kukkus veaga kokku?
Leht laeb ja laeb	Marsruut ei saatnud vastust.	Kas funktsioonis on `res.send` või `res.json`?
`SyntaxError`	Kirjaviga koodis.	Terminal näitab rea numbrit. Mine sinna.

Sõnastik

Sõna	Tähendus
klient	see, kes küsib. Meil brauser
server	programm, mis ootab päringuid ja vastab neile
päring	küsimus, mille klient serverile saadab
vastus	see, mille server tagasi saadab
Node.js	programm, mis käivitab JavaScripti ilma brauserita
Express	pakett, millega on lihtne serverit teha
pakett	kellegi teise kirjutatud kood, mille saad `npm install`-iga
JSON	andmed tekstina, näevad välja nagu JavaScripti massiivid ja objektid
port	ukse number, mille taga server arvutis ootab
marsruut	reegel: see aadress käivitab selle funktsiooni